

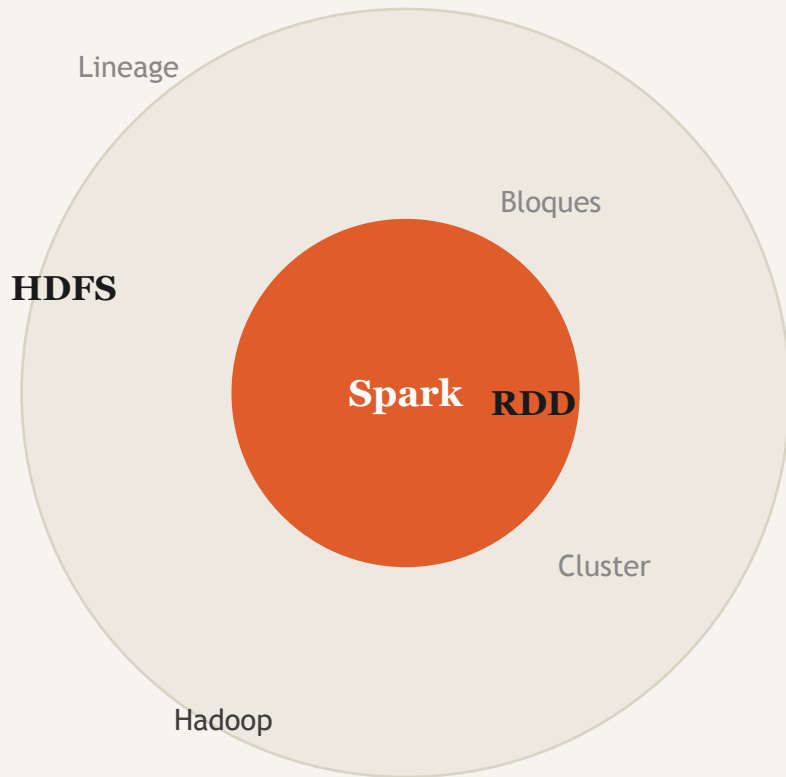


BIG DATA
FUNDAMENTOS

RDD & HDFS

Apache Spark · Hadoop

José Guerrero
Samuel Giraldo
Ricardo Hurtado



¿Qué veremos hoy?

01

¿Qué es HDFS?

Arquitectura del sistema de archivos distribuido

02

¿Qué es un RDD?

Concepto y propiedades del Resilient Distributed Dataset

03

Operaciones

Transformaciones y acciones para procesar datos

04

HDFS + RDD juntos

Integración y ejemplo real en PySpark

05

Casos de uso

Aplicaciones industriales en el mundo real

05

Hadoop Distributed File System

*"Diseñado para correr en hardware
almacenar conjuntos de
datos muy grandes de
forma confiable."*

NameNode

Gestiona el espacio de nombres y la metadata. No almacena datos reales.

DataNodes

Almacenan los bloques físicos. Reportan su estado al NameNode.

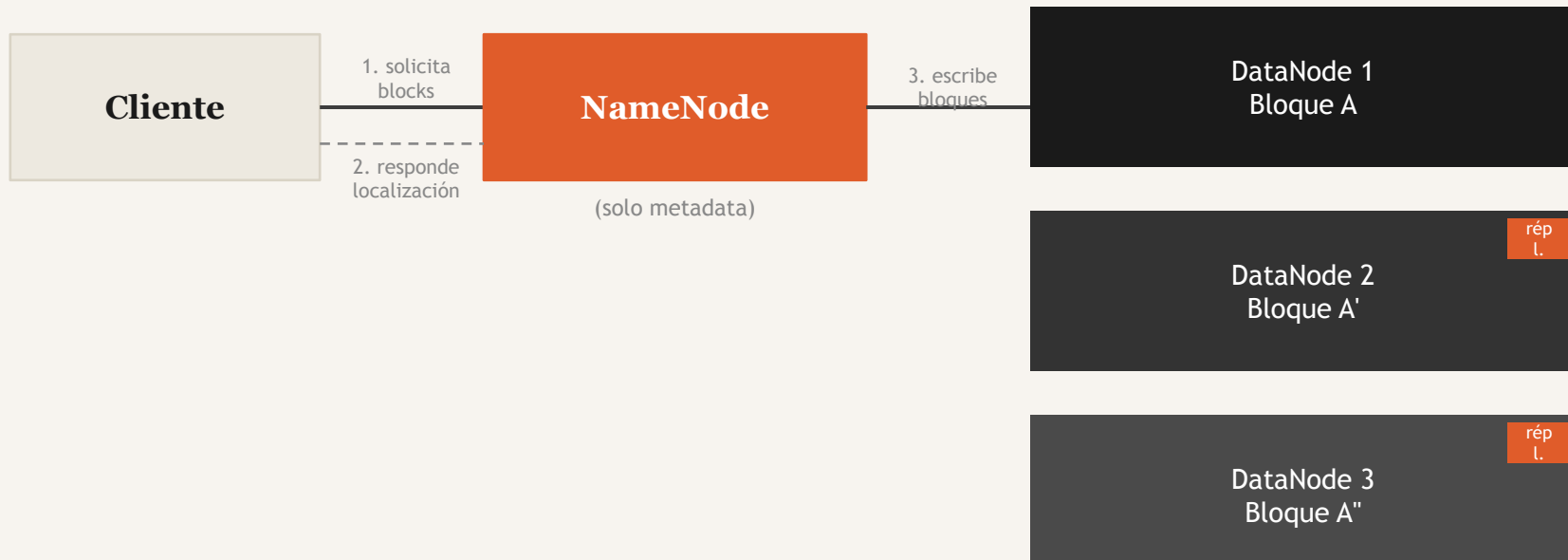
Bloques 128MB

Los archivos se parten en bloques de 128 MB distribuidos en el cluster.

Réplica ×3

Cada bloque se replica en 3 nodos diferentes para garantizar disponibilidad.

Flujo de escritura en HDFS



— Bloques de 128 MB por defecto

— NameNode solo guarda metadata

— Factor de replicación = 3

— Tolerancia a fallos automática

Resilient Distributed Dataset

R
Resilient

Recuperación automática ante fallos. Si un nodo cae, Spark reconstruye la partición perdida usando el grafo de linaje, sin intervención manual.

D
Distributed

Los datos se distribuyen en particiones a lo largo de todos los nodos del clúster, permitiendo procesamiento verdaderamente paralelo.

D
Dataset

Una colección de objetos de cualquier tipo – textos, números, tuplas – que Spark puede procesar con operaciones funcionales (map, filter, reduce).

Lo que hace especial a un RDD

01 Inmutabilidad

Nunca se modifican. Cada transformación genera un nuevo RDD.

03 Linaje

Spark recuerda el origen. Fallos se resuelven reconstruyendo.

05 Persistencia

Se pueden cachear en memoria con `persist()` o `cache()`.

02 Particionamiento

Divididos en particiones distribuidas procesadas en paralelo.

04 Evaluación perezosa

Nada se ejecuta hasta que una acción lo requiere.

06 Tolerancia a fallos

Particiones perdidas se regeneran automáticamente.



Transformaciones

Lazy – devuelven un nuevo RDD, no se ejecutan hasta que hay una acción

OPERACIÓN	DESCRIPCIÓN	EJEMPLO
<code>map(f)</code>	Aplica f a cada elemento	<code>rdd.map(x => x * 2)</code>
<code>filter(f)</code>	Filtra los que cumplen f	<code>rdd.filter(x => x > 5)</code>
<code>flatMap(f)</code>	Como map, aplana resultados	<code>rdd.flatMap(s => s.split(' '))</code>
<code>distinct()</code>	Elimina duplicados	<code>rdd.distinct()</code>
<code>union(rdd2)</code>	Une dos RDDs	<code>rdd1.union(rdd2)</code>
<code>groupByKey()</code>	Agrupar valores por clave	<code>pairs.groupByKey()</code>
<code>reduceByKey(f)</code>	Reduce valores por clave	<code>pairs.reduceByKey(_ + _)</code>
<code>sortBy(f)</code>	Ordena por función f	<code>rdd.sortBy(x => x)</code>

Acciones

Disparan la ejecución del DAG — devuelven datos al driver o escriben a disco

OPERACIÓN	DESCRIPCIÓN
<code>collect()</code>	Retorna todos los elementos al driver. Precaución con datos grandes.
<code>count()</code>	Número total de elementos en el RDD.
<code>first()</code>	Primer elemento del RDD.
<code>take(n)</code>	Los primeros n elementos como arreglo al driver.
<code>reduce(f)</code>	Agrega con f (debe ser asociativa y conmutativa).
<code>saveAsTextFile(path)</code>	Guarda el RDD como archivos de texto en HDFS.
<code>foreach(f)</code>	Ejecuta f en cada elemento (efectos secundarios).
<code>countByKey()</code>	Mapa con el conteo de cada valor único.

HDFS + RDD

HDFS

Datos persistentes
en bloques

SparkContext

Crea particiones
desde HDFS

RDD

Procesamiento
en memoria

Resultado

Escribe de vuelta
a HDFS

PySpark

```
from pyspark import SparkContext
sc = SparkContext('local[*]', 'Demo')

# Leer desde HDFS → crea un RDD
rdd = sc.textFile(
    'hdfs://namenode:9000/ventas.csv')

# Transformaciones (lazy)
resultado = (
    rdd
    .filter(lambda l: 'ERROR' not in l)
    .map(lambda l: l.split(','))
    .reduceByKey(lambda a, b: a + b)
)

# Acción → escribe a HDFS
resultado.saveAsTextFile('hdfs://.../output')
```



¿Dónde se usa HDFS + Spark?

• STREAMING

Netflix · Spotify

Sistemas de recomendación analizando billones de eventos de escucha y visualización.

◆ FINANZAS

Bancos · Fintechs

Detección de fraude en tiempo real sobre millones de transacciones simultáneas.

◆ E-COMMERCE

Amazon · Mercado Libre

Análisis de logs, comportamiento de compra y precios dinámicos en escala global.

◇ CIENCIA

Bioinformática · CERN

Secuenciación de genomas y análisis de colisiones de partículas — terabytes diarios.

◇ TELECOMUNICACIONES

Claro · Movistar

Análisis de tráfico de red y calidad de servicio para millones de usuarios.

□ IOT / AUTOS

Tesla · Flota Logística

Procesamiento de streams de sensores en tiempo casi-real para flotas conectadas.



Diferencias esenciales

ASPECTO	HDFS	RDD
Tipo	Sistema de archivos distribuido	Abstracción de datos en memoria
Propósito	Almacenamiento persistente	Procesamiento paralelo
Dónde vive	Disco de los DataNodes	RAM del clúster (o disco si no cabe)
Mutabilidad	Archivos inmutables por bloques	Inmutable por diseño funcional
Velocidad	Alta para lecturas secuenciales	100× más rápido que disco en memoria
Uso típico	Almacenar datos crudos (TB / PB)	Transformar y analizar datos

Se complementan: HDFS almacena, Spark/RDD procesa.

Resumen clave

HDFS
almacena datos
a escala masiva

RDD
procesa datos
en paralelo

Juntos
son el estándar
de la industria

01

HDFS divide archivos en bloques de 128 MB replicados en 3 nodos distintos para garantizar disponibilidad.

02

Los RDDs son inmutables y resilientes — si un nodo falla, Spark reconstruye la partición desde el linaje.

03

Las transformaciones son lazy: Spark optimiza el plan de ejecución completo antes de correr.

04

`reduceByKey()` es más eficiente que `groupByKey()` porque reduce antes de mover datos por la red.

05

Cachear RDDs con `persist()` evita releer HDFS en iteraciones repetidas (machine learning, grafos).